

Predictive Statistical Problems Assignment Part 1-30%

Bruno de Arantes Leite Sassi

Prof Zeeshan Ahmad

University of Niagara Falls

Master of Data Analytics

DAMO 510-1 – Predictive Analytics

May, 28th 2025

PART I . Using 'statsmodels' package for on interpretation and explanation

1. Original Model: (30 points)

a. Interpret the model (coefficients, p-values, R-sq and Fisher) (10 points)

The original model achieves an R-squared value of 0.712, meaning approximately 71.2% of the variance in house prices is explained by the predictors: square footage, age, and number of bedrooms. The adjusted R-squared (0.711) confirms that model complexity does not lead to overfitting. The F-statistic is 819.1 with a p-value effectively zero ($p < 0.0001$), indicating the model is highly statistically significant overall. Interpreting the coefficients, each additional square foot adds about \$95.36 to the house price, each additional year of age reduces price by approximately \$789.34, and each additional bedroom increases price by around \$20,010. All variables have p-values less than 0.001, demonstrating strong statistical significance. These results suggest the model is meaningful, though further diagnostic review is needed.

b. What problem(s) can you identify, and which transformation would be most appropriate to address these issue(s)? (10 points)

The model shows clear signs of heteroskedasticity. The residuals vs. fitted plot reveals a fanning pattern, where residual spread increases with fitted values. Moreover, the Q-Q plot indicates non-normality, with heavy tails and skewness. The condition number (8,180) also raises concerns about multicollinearity or numerical instability. To address these issues, a log transformation is appropriate. This often reduces variance, improves normality, and helps stabilize relationships.

c. Interpret the variance of the residuals? (10 points)

The variance of the residuals in the original model is not constant — it increases with fitted values, indicating heteroscedasticity. Additionally, the Jarque-Bera and

Omnibus tests confirm non-normality in the residual distribution. These limitations suggest the need for a transformation to meet model assumptions and improve accuracy.

2. Log-transformed Model: (30 points)

a. When are log-transformed variables sometimes more suitable? (10 points)

Log-transformed variables are more appropriate when the dependent variable exhibits right skewness, has large variance, or contains outliers. Applying a log transformation can help improve the linearity of relationships between variables, reduce heteroskedasticity, and facilitate interpretation of coefficients.

b. Interpret the model (coefficients, p-values, R-sq and Fisher)? (10 points)

The log-transformed model yields a slightly lower R-squared of 0.696 and adjusted R-squared of 0.695. Although these values are below those of the original model, the F-statistic remains high at 760.1 with a p-value approaching zero, confirming strong overall model significance. After transforming the dependent variable ($\log(\text{price})$), the interpretation of coefficients shifts: each additional square foot increases the log of the price by 0.4422 units, while each additional year of age decreases it by 0.0021 units. Each additional bedroom increases $\log(\text{price})$ by 0.0504. All variables retain p-values below 0.001, indicating high significance.

c. Interpret the variance of the residuals? (10 points)

This transformation improves the model's behavior. The residuals vs. fitted plot shows more consistent spread, indicating reduced heteroskedasticity. The Q-Q plot is much closer to a straight line, suggesting that residuals approximate a normal distribution more closely. Additionally, the kurtosis and skewness values are much lower (3.627 and -0.462, respectively), which aligns more closely with the assumptions

of linear regression. The condition number is also greatly reduced (to 584), lessening previous concerns regarding multicollinearity or numerical precision. Overall, the log transformation enhances the robustness and reliability of the regression model.

PART II. Using 'sklearn.model_selection' package with focus on prediction

3. Compare the R^2 values between the original and transformed models. Why might the R^2 values not be directly comparable between these models? What additional metrics should we consider? (20 points)

The R-squared values indicate how well each model explains the variability in the data. The original model shows an R^2 of approximately 0.63, while the log-transformed and square root transformed models have higher R^2 values of about 0.69 and 0.69 respectively, suggesting improved fit after transformation. However, these R^2 values are not directly comparable because they are calculated on different scales: the original model uses the raw price, whereas the others use transformed versions of price (logarithm or square root). Thus, a higher R^2 in the transformed models does not necessarily mean a better prediction in the original scale. To properly evaluate and compare models, metrics like Root Mean Squared Error (RMSE) should also be considered. RMSE measures the average prediction error magnitude in the original units of the dependent variable, which helps assess practical performance. In this case, the original model's RMSE is very large (~60,598), the log-transformed model has a drastically smaller RMSE (~0.13) due to the scale of the transformed target, and the square root transformation offers an intermediate RMSE (~42). These differences illustrate why relying solely on R^2 can be misleading when comparing models with differently scaled targets. Therefore, it is important to also consider other metrics such as Mean Absolute Error (MAE), cross-validated errors, residuals, among other metrics for more comprehensive understanding of model performance.

4. Using the Python code provided, add a new transformation to the model using the Box-Cox transformation for the price variable. Compare its performance with the log and square root transformations. When would you recommend using Box-Cox over a simple log transformation? (20 points)

The Box-Cox transformation flexibly determines the optimal power transformation to normalize the target variable and stabilize variance. In this case, the optimal lambda (λ) found was approximately -0.41, which differs from the fixed transformations of log ($\lambda = 0$) and square root ($\lambda = 0.5$). The regression model trained on the Box-Cox transformed price achieved an R-squared of 0.614 and an RMSE of 0.00072 (in the Box-Cox transformed scale). While the R-squared is somewhat lower compared to the log and square root models, the very low RMSE indicates tight prediction errors on the transformed scale. Residual diagnostics also suggest improved homoscedasticity and better adherence to model assumptions.

The advantage of the Box-Cox method lies in its ability to adapt the transformation to the data rather than assuming a fixed transformation type, making it especially useful when the distribution of the target variable is complex or does not fit well with standard transformations. Given the negative lambda, the transformation here is closer to an inverse power, highlighting the specific skewness in the dataset. Therefore, Box-Cox is recommended when exploratory data analysis shows that simpler transformations are insufficient or when fine-tuning model fit is a priority.

Appendix A

Part I results for data processing using “statsmodels” package.

Figure A1. Original Model Results

Original Model Results:

OLS Regression Results

Dep. Variable:	price	R-squared:	0.712
Model:	OLS	Adj. R-squared:	0.711
Method:	Least Squares	F-statistic:	819.1
Date:	Sun, 25 May 2025	Prob (F-statistic):	2.63e-268
Time:	22:19:48	Log-Likelihood:	-12311.
No. Observations:	1000	AIC:	2.463e+04
Df Residuals:	996	BIC:	2.465e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	1.912e+05	6487.386	29.473	0.000	1.78e+05	2.04e+05
square_footage	95.3590	2.066	46.158	0.000	91.305	99.413
age	-789.3418	118.408	-6.666	0.000	-1021.699	-556.985
bedrooms	2.001e+04	1196.268	16.726	0.000	1.77e+04	2.24e+04
Omnibus:	170.196	Durbin-Watson:	1.976			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1171.862			
Skew:	-0.581	Prob(JB):	3.41e-255			
Kurtosis:	8.174	Cond. No.	8.18e+03			

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 8.18e+03. This might indicate that there are strong multicollinearity or other numerical problems.

Figure A2. Log-Transformed Model Results

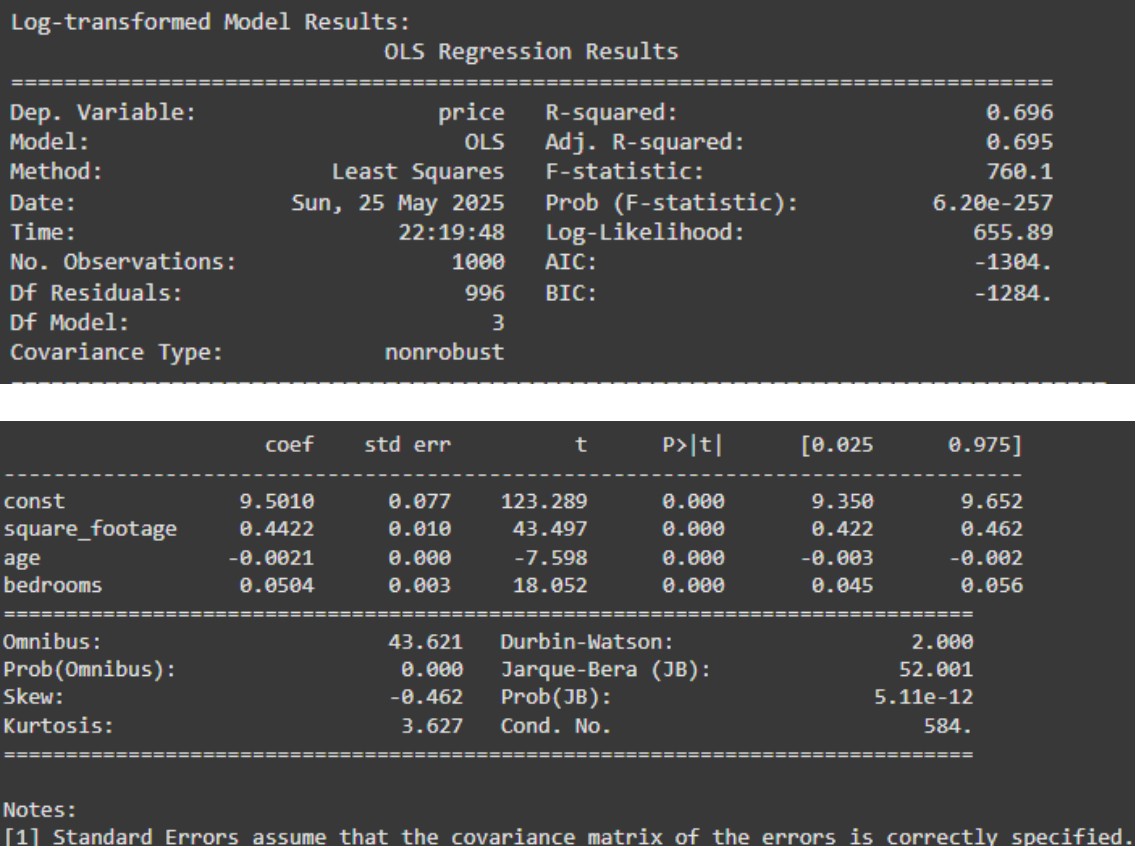


Figure A3. Original Model and Log Transformed Model Residual and Q-Q Plots

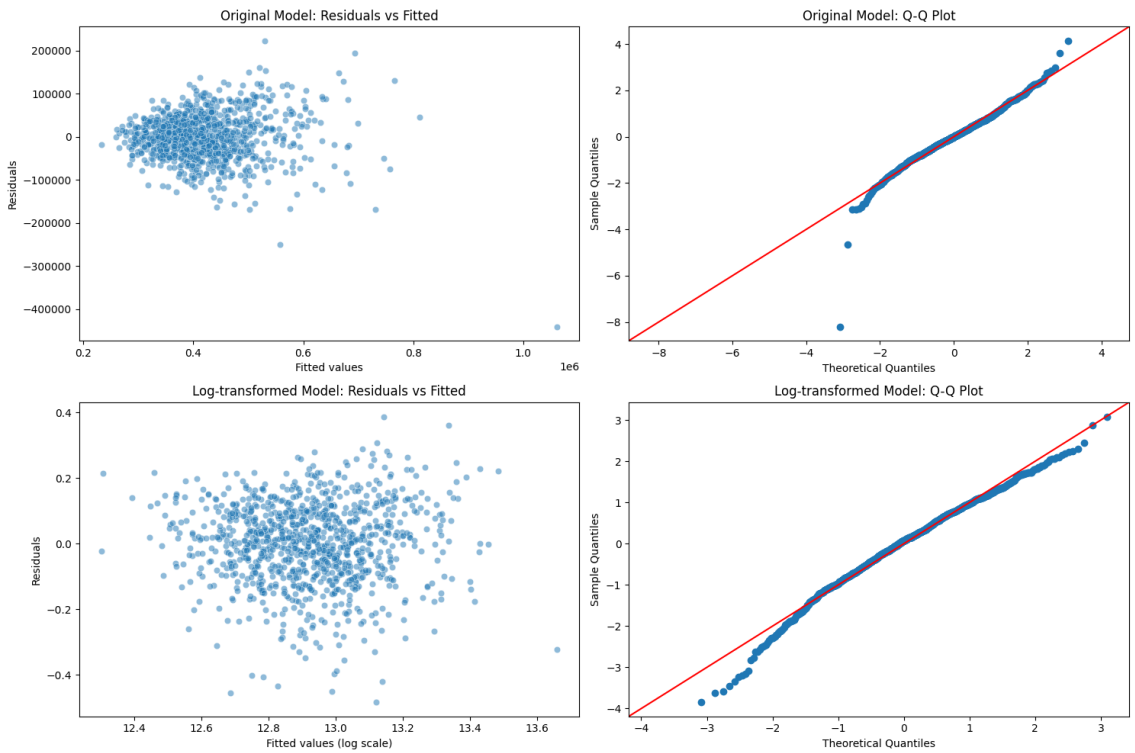


Figure A4. VIF and Breusch-Pagan Results for the Original and Log-transformed Models

```
VIF for Original Model:
      Variable      VIF
0      const  14.506181
1 square_footage  1.000278
2      age  1.003549
3   bedrooms  1.003271

VIF for Log-transformed Model:
      Variable      VIF
0      const  375.076563
1 square_footage  1.000576
2      age  1.003846
3   bedrooms  1.003279

Breusch-Pagan test for Original Model:
LM stat      1.854479e+02
LM p-value    5.873136e-40
F stat        7.558597e+01
F p-value     4.731483e-44
dtype: float64

Breusch-Pagan test for Log-transformed Model:
LM stat      5.886255e+01
LM p-value    1.028533e-12
F stat        2.076463e+01
F p-value     4.702575e-13
dtype: float64
```

Appendix B

Part II results for data processing using “sklearn.model_selection” package.

Figure B1. Model Performance Comparison

Model Performance Comparison with Box-Cox added:			
	Model	R-squared	RMSE
0	Original	0.629619	60598.041947
1	Log-transformed	0.689352	0.128276
2	Square root	0.686921	41.981625
3	Box-Cox	0.614461	0.000724

Figure B2. Original Model: Residuals vs Fitted Values

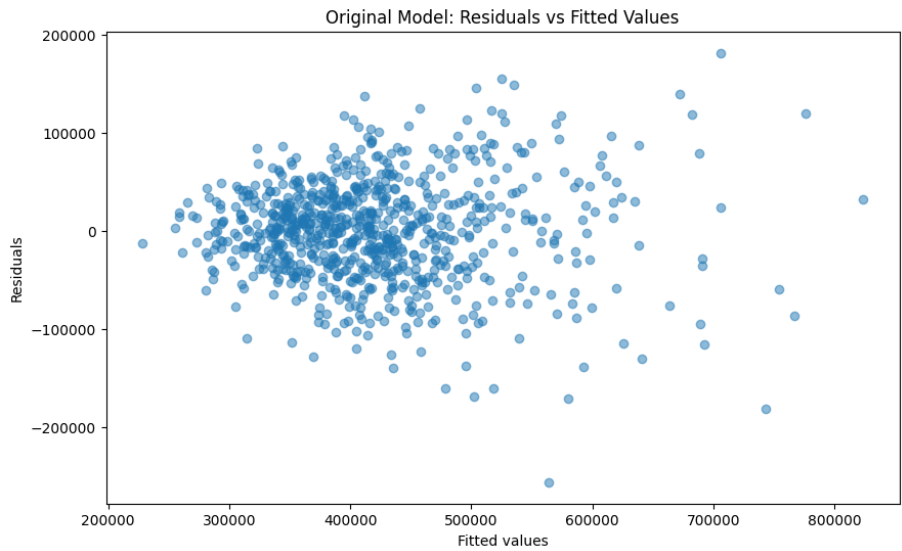


Figure B3. Log-Transformed Model: Residuals vs Fitted Values

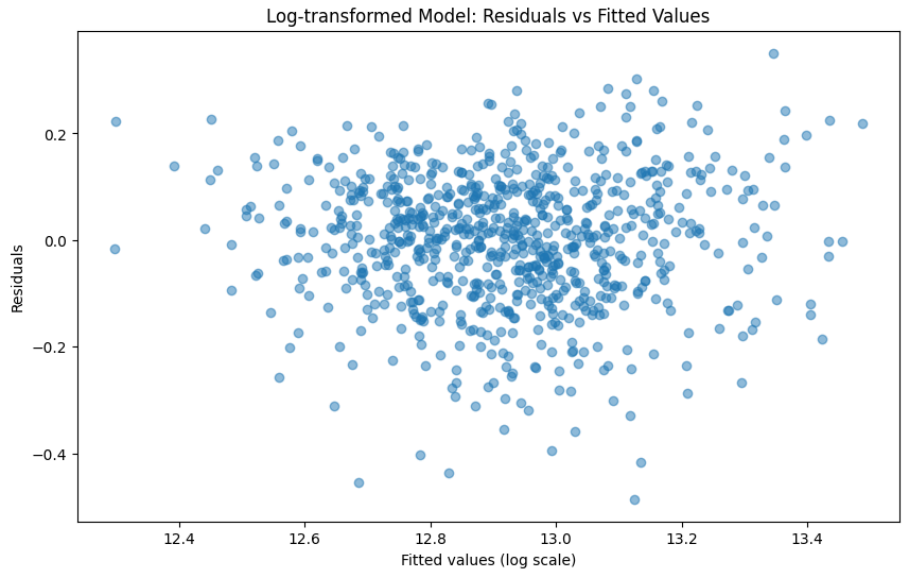


Figure B4. Sqrt-Transformed Model: Residuals vs Fitted Values

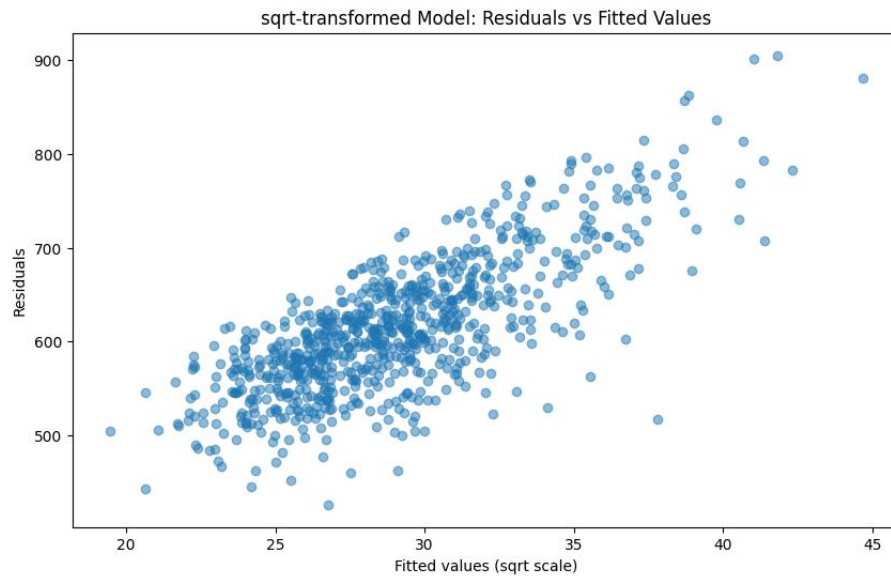


Figure B5. Box-Cox Model: Residuals vs Fitted Values

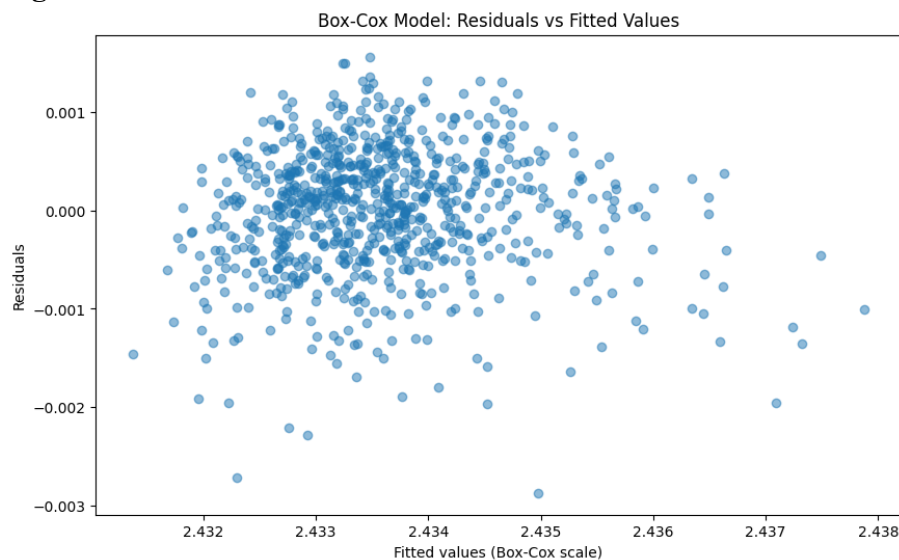


Table B1. Additional code used in part II

Function	Code
Plot Sqrt residual chart	<pre># Plot residuals vs fitted values for square-root-transformed model plt.figure(figsize=(10, 6)) y_pred_train_sqrt = model2.predict(X_train_sqrt) residuals_sqrt = y_train_sqrt - y_pred_train_sqrt plt.scatter(y_pred_train_sqrt, residuals_sqrt, alpha=0.5) plt.xlabel('Fitted values (sqrt scale)') plt.ylabel('Residuals') plt.title('sqrt-transformed Model: Residuals vs Fitted Values') plt.show()</pre>
	<pre>import numpy as np import pandas as pd import matplotlib.pyplot as plt from sklearn.model_selection import train_test_split from sklearn.linear_model import LinearRegression</pre>

	<pre> from sklearn.metrics import r2_score, mean_squared_error from scipy.stats import boxcox # (Given code was already runned) # Box-Cox transformation on target variable 'price' # Box-Cox requires strictly positive data, which price is assumed to be. # Find optimal lambda on training target y_train_positive = y_train + 1e-6 # Shift slightly if zero values exist (just in case) y_train_boxcox, fitted_lambda = boxcox(y_train_positive) # Transform test target using the same lambda def boxcox_transform(y, lmbda): if lmbda == 0: return np.log(y) else: return (np.power(y, lmbda) - 1) / lmbda y_test_positive = y_test + 1e-6 y_test_boxcox = boxcox_transform(y_test_positive, fitted_lambda) # Fit Linear Regression on original predictors (without transforming predictors for Box-Cox) model_boxcox = LinearRegression() model_boxcox.fit(X_train, y_train_boxcox) y_pred_boxcox = model_boxcox.predict(X_test) # Compute R-squared and RMSE on Box-Cox scale r2_boxcox = r2_score(y_test_boxcox, y_pred_boxcox) rmse_boxcox = np.sqrt(mean_squared_error(y_test_boxcox, y_pred_boxcox)) # Add Box-Cox results to the results DataFrame results = pd.DataFrame({ 'Model': ['Original', 'Log-transformed', 'Square root', 'Box-Cox'], 'R-squared': [r2_1, r2_2, r2_3, r2_boxcox], 'RMSE': [rmse_1, rmse_2, rmse_3, rmse_boxcox] }) print("\nModel Performance Comparison with Box-Cox added:") print(results) # Plot residuals vs fitted values for Box-Cox model plt.figure(figsize=(10, 6)) y_pred_train_boxcox = model_boxcox.predict(X_train) residuals_boxcox = y_train_boxcox - y_pred_train_boxcox plt.scatter(y_pred_train_boxcox, residuals_boxcox, alpha=0.5) plt.xlabel('Fitted values (Box-Cox scale)') plt.ylabel('Residuals') plt.title('Box-Cox Model: Residuals vs Fitted Values') plt.show() </pre>
Optimal Box-Cox lambda	<pre> print(f'Optimal Box-Cox lambda: {fitted_lambda:.4f}') </pre>